"It's Like a TA That Never Sighs": Novices' Mental Models of Code Assistants in CS1

Maja Lindgren, Kofi Mensah

In Proceedings of the 30th ACM Conference on Innovation and Technology in Computer Science Education V. 1 (ITiCSE '25), pp. 88-94.

Abstract

We interviewed 22 CS1 students about how they understand and use an AI code assistant permitted in their course. Students held three recurring models of the assistant: an oracle that knows the answer, a rubber duck that helps them think, and a shortcut that saves time on parts they consider unimportant. Models shifted within a single session, often after a failed run. Students who described the assistant as a rubber duck reported asking later and asking narrower questions. We discuss implications for when and how assistants should respond to novices.

1 Introduction

How students use an assistant depends on what they think it is. A student who treats it as an oracle asks for the answer; one who treats it as a sounding board asks whether their plan makes sense. We report an interview study of these mental models.

2 Method

We recruited 22 students (11 women, 10 men, 1 non-binary; 15 with no prior programming) from a CS1 course in which an assistant was permitted for homework. Semi-structured interviews (35-50 minutes) included a think-aloud on a short debugging task with the assistant available. Two researchers coded transcripts with reflexive thematic analysis.

3 Three models

Model	Students who described it (of 22)	Typical first prompt
Oracle	17	"write a function that ..."
Rubber duck	9	"is my approach right if ..."
Shortcut	14	"fix the syntax in this"

Most students described more than one model. P7 explained: "When it's the hard part I want to figure it out, so I just ask if I'm on track. When it's string formatting I just want it gone." The title quote came from P15, who said the assistant was like a TA that "never sighs when you ask the same thing twice".

4 Shifts within a session

In the think-aloud, 13 students changed how they addressed the assistant after a failed run. The most common shift (9 students) was from duck to oracle: after one or two failures, they pasted the error and asked for corrected code. Students described this moment as running out of patience rather than running out of ideas.

Rubber-duck users waited longer before their first prompt (median 6.5 vs 2.0 minutes) and wrote narrower prompts that referenced specific lines.

5 Implications

Assistants could support the duck model explicitly, for example by asking a student to state their plan before giving code, and by noticing the duck-to-oracle shift after repeated failures, which may be the moment a human check-in helps most.

6 Limitations

Self-report and a single think-aloud task. Our sample over-represents students who volunteer for interviews.

References

[1] Wood, D., Bruner, J. S., and Ross, G. The role of tutoring in problem solving. Journal of Child Psychology and Psychiatry 17, 2 (1976), 89-100.

[2] Aleven, V. et al. Help seeking and help design in interactive learning environments. Review of Educational Research 73, 3 (2003), 277-320.